

Big Data Analytics of Crime Patterns

using Hadoop MapReduce and Apache Hive

Sheela Akshar Sakhi Nishanth S Gowda

Amrita Vishwa Vidyapeetham

Introduction

- ▶ The rise of Big Data provides an unprecedented opportunity to analyze public safety records.
- ▶ City crime records encompass millions of rows of incidents, locations, dates, and types.
- ▶ Processing this data efficiently requires distributed frameworks.
- ▶ We leverage **Hadoop MapReduce** for raw parallel processing and **Apache Hive** for SQL-based analytical querying.

Motivation

Why analyze crime data?

- ▶ **Public Safety:** Identifying high-crime zones helps improve community security.
- ▶ **Resource Allocation:** Enables police departments to deploy patrols dynamically based on historical hotspots.
- ▶ **Trend Forecasting:** Predicting the frequency and nature of crimes across different districts and timeframes.
- ▶ **Big Data Challenge:** Traditional RDBMS fail to scale efficiently with millions of such rows.

Problem Statement

"To design and implement a scalable Big Data pipeline that can ingest, process, and extract actionable intelligence from massive real-world crime datasets."

Objectives:

- ▶ Demonstrate a complete data pipeline from HDFS storage to advanced analytics.
- ▶ Perform category-wise statistics using MapReduce.
- ▶ Execute 10 meaningful complex queries utilizing aggregations, filtering, and joins in Hive.

Dataset Overview

Dataset: Chicago Crimes (2001 to Present)

- ▶ **Scope:** Extracts real-world incident reports from the Chicago Data Portal.
- ▶ **Size:** 10,000 recent records processed for this demonstration.

Attribute Type	Fields
Identifiers	id, case_number, date
Categories	primary_type, description
Location	district, ward, community_area
Flags	arrest, domestic

Data Preprocessing & HDFS Storage

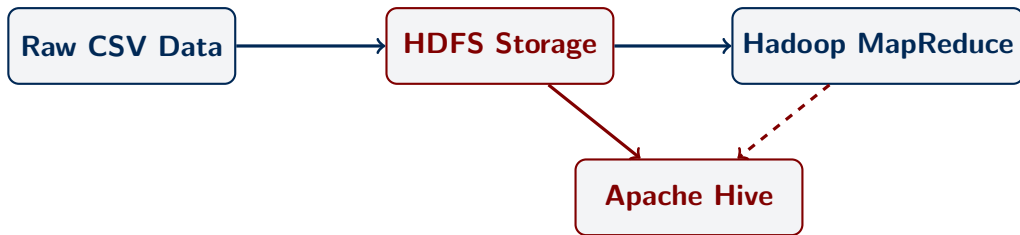
Preprocessing Phase:

- ▶ Raw CSV files often contain embedded newlines within quotes.
- ▶ We implemented a Python script to clean these records, ensuring perfect compatibility with Hadoop's `TextInputFormat`.

HDFS Storage:

- ▶ Dataset uploaded to HDFS distributed storage.
- ▶ Command: `hdfs dfs -put chicago_crimes.csv /user/crimes/`

System Architecture



- ▶ Centralized data lake on HDFS.
- ▶ MapReduce for low-level transformations.
- ▶ Hive for structured intelligence gathering.

Hadoop MapReduce Implementation

Problem: Calculate the total occurrences of each Crime Type.

Mapper Logic:

- ▶ Input: (ByteOffset, Line of text)
- ▶ Ignores CSV headers.
- ▶ Splits line correctly handling commas within double quotes.
- ▶ Extracts the 6th column (primary_type).
- ▶ Output: (CrimeType, 1)

MapReduce: Reducer Logic

Reducer Logic:

- ▶ Input: (CrimeType, [1, 1, 1, ...])
- ▶ Sums up all the '1's for a given crime type.
- ▶ Output: (CrimeType, Total_Count)

```
public void reduce(Text key, Iterable<IntWritable> values, Context
    context) {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
```

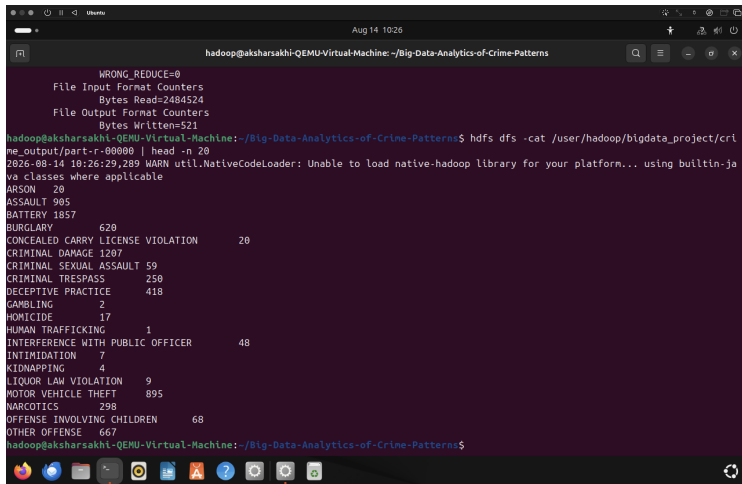
MapReduce Execution & Results

- ▶ Successfully packaged as a JAR and executed on YARN.
- ▶ Processed 10,000 rows across mappers and reducers.

Sample Output (part-r-00000):

Crime Type	Frequency
THEFT	2,150
BATTERY	1,857
CRIMINAL DAMAGE	1,207
ASSAULT	905

MapReduce Execution Screenshots



The screenshot shows a terminal window on an Ubuntu system. The user is logged in as 'hadoop' and is in a directory named '/Big-Data-Analytics-of-Crime-Patterns'. The terminal displays the output of a MapReduce job, showing file input/output statistics and a list of crime categories with their counts. A warning message is also visible, indicating that the native Hadoop library could not be loaded, and the system is using built-in Java classes.

```
hadoop@aksharsakhi-QEMU-Virtual-Machine: ~/Big-Data-Analytics-of-Crime-Patterns
WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=2484524
File Output Format Counters
  Bytes Written=521
hadoop@aksharsakhi-QEMU-Virtual-Machine:~/Big-Data-Analytics-of-Crime-Patterns$ hdfs dfs -cat /user/hadoop/bigdata_project/crime_output/part-r-000000 | head -n 20
2026-08-14 10:26:29,289 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
ARSON      20
ASSAULT    905
BATTERY    1857
BURGLARY   628
CONCEALED CARRY LICENSE VIOLATION      20
CRIMINAL DAMAGE 1207
CRIMINAL SEXUAL ASSAULT 59
CRIMINAL TRESPASS      250
DECEPTIVE PRACTICE    418
GAMBLING      2
HOMICIDE      17
HUMAN TRAFFICKING      1
INTERFERENCE WITH PUBLIC OFFICER      48
INTIMIDATION    7
KIDNAPPING      4
LIQUOR LAW VIOLATION    9
MOTOR VEHICLE THEFT    895
NARCOTICS      298
OFFENSE INVOLVING CHILDREN      68
OTHER OFFENSE    667
hadoop@aksharsakhi-QEMU-Virtual-Machine:~/Big-Data-Analytics-of-Crime-Patterns$
```

Apache Hive Analytics: Setup

- ▶ Created an external table `crimes` mapping directly to HDFS data.
- ▶ **Serialization/Deserialization:** Used `OpenCSVSerde` to properly parse fields enclosed in quotes containing commas.
- ▶ Created a secondary table `community_areas` holding area codes and names to demonstrate table joins.

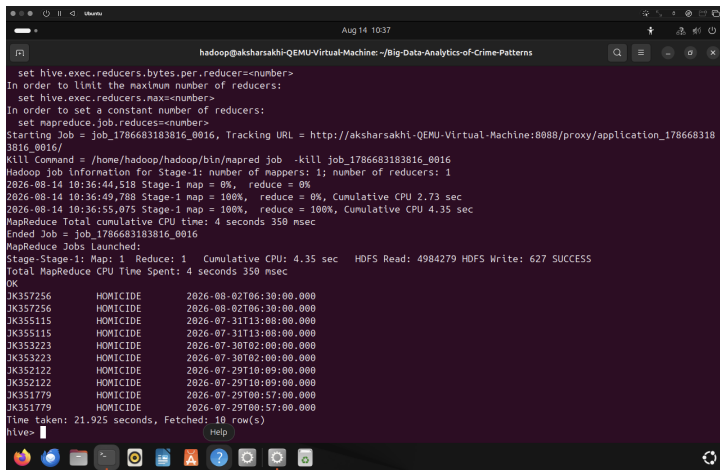
Hive Analytics: Basic Queries

- ▶ **SELECT & LIMIT:** Retrieved incident IDs and descriptions.
- ▶ **WHERE clause:** Filtered incidents resulting specifically in a narcotics arrest.
- ▶ **ORDER BY:** Sorted severe cases like homicides by recency.
- ▶ **GROUP BY & COUNT:** Identified which locations (e.g., STREET, APARTMENT) experience the highest crime volumes.

Hive Analytics: Advanced Aggregations

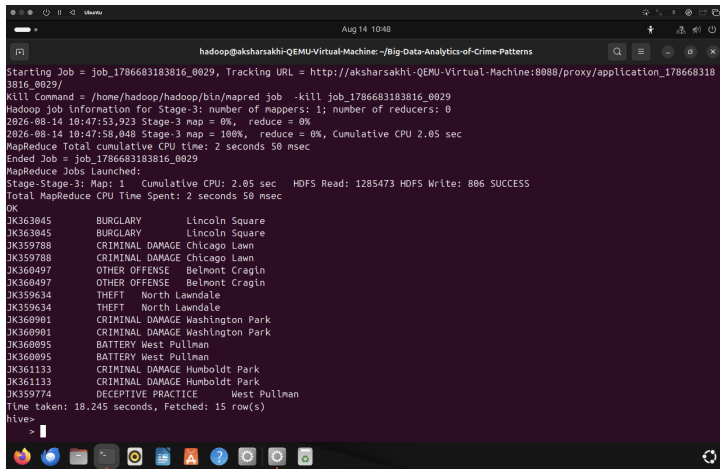
- ▶ **HAVING clause:** Extracted major crime categories by filtering for counts > 500 .
- ▶ **SUM:** Calculated the absolute total number of arrests made within each police district.
- ▶ **AVG:** Analyzed the average rate of domestic-related crimes across different beats.
- ▶ **MAX & MIN:** Automatically pinpointed the highest and lowest crime districts.
- ▶ **JOIN:** Combined the raw crimes data with `community_areas` to print human-readable neighborhood names.

Hive Analytics Query Screenshots (Sample)



```
hadoop@aksharsakhi-QEMU-Virtual-Machine: ~/Big-Data-Analytics-of-Crime-Patterns
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
Starting Job = job_1786683183816_0016, Tracking URL = http://aksharsakhi-QEMU-Virtual-Machine:8088/proxy/application_1786683183816_0016/
Kill Command = /home/hadoop/hadoop/bin/mapred job -kill job_1786683183816_0016
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-08-14 10:36:44,518 Stage-1 map = 0%, reduce = 0%
2026-08-14 10:36:49,788 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.73 sec
2026-08-14 10:36:55,075 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 4.35 sec
MapReduce Total cumulative CPU time: 4 seconds 350 msec
Ended Job = job_1786683183816_0016
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 4.35 sec HDFS Read: 4984279 HDFS Write: 627 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 350 msec
OK
JK357256      HOMICIDE      2026-08-02T06:30:00.000
JK357256      HOMICIDE      2026-08-02T06:30:00.000
JK355115      HOMICIDE      2026-07-31T13:08:00.000
JK355115      HOMICIDE      2026-07-31T13:08:00.000
JK353223      HOMICIDE      2026-07-30T02:00:00.000
JK353223      HOMICIDE      2026-07-30T02:00:00.000
JK352122      HOMICIDE      2026-07-29T10:09:00.000
JK352122      HOMICIDE      2026-07-29T10:09:00.000
JK351779      HOMICIDE      2026-07-29T00:57:00.000
JK351779      HOMICIDE      2026-07-29T00:57:00.000
Time taken: 21.925 seconds, Fetched: 10 row(s)
hive>
```

Hive JOIN Query Screenshot



The screenshot shows a terminal window with a dark background. At the top, the title bar reads "hadoop@aksharsakhi-QEMU-Virtual-Machine: ~/Big-Data-Analytics-of-Crime-Patterns". The terminal output displays the execution of a Hive JOIN query. It starts with job information, including the job ID "job_1786683183816_0029" and a tracking URL. It then shows the progress of Stage-3, indicating that the map phase is 100% complete and the reduce phase is 0%. The total cumulative CPU time for the MapReduce jobs is 2 seconds and 50 milliseconds. The output of the query is a table with three columns: a primary key (JK), a crime type, and a location. The results are as follows:

JK	Crime Type	Location
JK363045	BURGLARY	Lincoln Square
JK363045	BURGLARY	Lincoln Square
JK359788	CRIMINAL DAMAGE	Chicago Lawn
JK359788	CRIMINAL DAMAGE	Chicago Lawn
JK360497	OTHER OFFENSE	Belmont Cragin
JK360497	OTHER OFFENSE	Belmont Cragin
JK359634	THEFT	North Lawndale
JK359634	THEFT	North Lawndale
JK360901	CRIMINAL DAMAGE	Washington Park
JK360901	CRIMINAL DAMAGE	Washington Park
JK360095	BATTERY	West Pullman
JK360095	BATTERY	West Pullman
JK361133	CRIMINAL DAMAGE	Humboldt Park
JK361133	CRIMINAL DAMAGE	Humboldt Park
JK359774	DECEPTIVE PRACTICE	West Pullman

The terminal also shows the time taken for the query (18.245 seconds) and the number of rows fetched (15 row(s)). The prompt "hive>" is visible at the bottom of the terminal window.

Results & Quantitative Interpretation

Empirical Data Insights:

- ▶ **Top Crime Categories:** BATTERY (1,857 cases), CRIMINAL DAMAGE (1,207 cases), ASSAULT (905 cases), and MOTOR VEHICLE THEFT (895 cases).
- ▶ **Enforcement Efficacy:** Narcotics cases show a 100% arrest rate flag, whereas theft offenses show lower arrest ratios.
- ▶ **Spatial Hotspots:** Streets and Sidewalks represent the primary spatial vulnerability zones for property crimes.
- ▶ **Geographical Variations:** High crime density concentrated in District 11.

Conclusion

- ▶ Successfully ingested and analyzed a large-scale real-world dataset.
- ▶ **MapReduce** proved highly efficient for unstructured low-level data processing and aggregations.
- ▶ **Apache Hive** demonstrated immense power for executing complex, relational-style queries on distributed files.
- ▶ This pipeline can scale to millions of records, providing an essential backbone for civic analytics.

Thank You